#5 Removes the entry from the hash table.
#5 Returns true to flag success.

There are still a couple of methods in the API that we haven't discussed yet: `get(key)` and `getSize()`. Their implementation, though, is trivial, because they are just wrappers for the homonymous methods in hash tables, at least if we keep values in the hash table directly; if, instead, we keep values in the linked list, and the hash table has pointers to list nodes, then in `get` we need to resolve this further indirection.

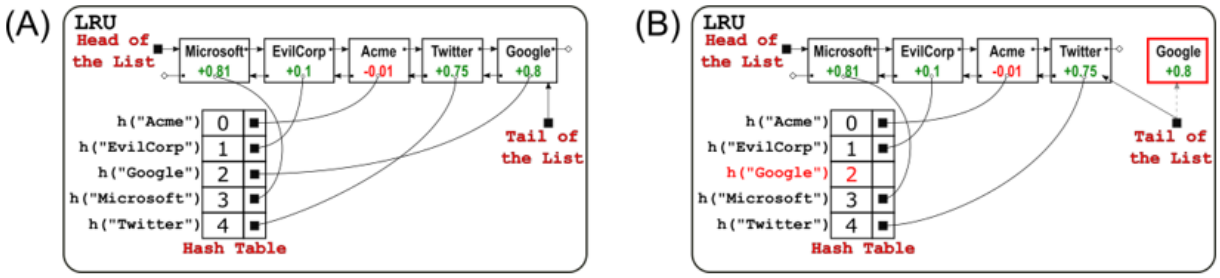


Figure 7.8 Removal of the LRU entry (will be followed by an add on miss, like the one in figure 7.6).
(A) The initial state of a full cache.
(B) The pointer to the tail of the list is updated, as well as the links from/to the second-to-last node. The corresponding entry in the hash table is also removed, so the node for the entry is not referenced from the cache anymore (depending on the language it might be garbage collected, or we need to manually destroy it and release the memory it uses).

Listing 7.4 LRU Cache get

```
function LRUCache::get(key)
```